ISLS Extension Phase 3 v1.0.0

C18: Hierarchical Multi-Scale Observation Layer

Hypercube Universes, Dimensional Bridges,
and Scale Ladders for the
Intelligent Semantic Ledger Substrate

Absorbing CubeTube Kybernetic Core
Architecture Patterns

Sebastian Klemm

16 March 2026

Abstract

This document specifies `isls-scale` (C18), a hierarchical multi-scale observation layer for ISLS. The crate enables the system to observe, analyze, and crystallize patterns simultaneously across three scales—micro (individual entities), meso (synchronization clusters), and macro (system-of-clusters)—using a unified formalism of Hypercube Universes, Dimensional Bridges, and Scale Ladders.

Each scale is modeled as an independent state space (Hypercube Universe) with its own transition dynamics and policy. Bridges couple Hypercubes at the same scale (mesoscopic field coupling). Ladders connect Hypercubes across scales via Lift and Projection operators (microscopic–macroscopic linkage). Crystals produced at any scale carry a hierarchical topological signature referencing structures at all three scales.

The architecture absorbs the StateSpace, Policy, Transition, Dynamics, HyperBounds, and Blueprint abstractions from CubeTube Kybernetic Core, adapted to ISLS's deterministic, append-only, replay-verified substrate.

Status: Implementation Specification.

Parent: ISLS v1.0.0 + Extensions Phase 1 + Phase 2.
Absorbs: CubeTube Kybernetic Core (architectural patterns).
Depends on: C16 (`isls-topology`) for spectral clustering and Kuramoto synchronization.

Contents

I Foundations 5

1 Purpose 5

2 Axioms 5

3 Relationship to Existing Crates 5

II Scale Architecture 5

4 The Three Scales 6

5 Hypercube Universe 6

6 Transitions and Dynamics 7

III Bridges and Ladders 7

7 Dimensional Bridges 7

8 Scale Ladders 8

8.1 Micro → Meso Ladder 8

8.2 Meso → Macro Ladder 9

8.3 Ladder Consistency 9

IV Multi-Scale Engine Integration 9

9 Per-Scale Macro-Step 9

10 Cluster Identification 10

11 Multi-Scale Crystallization 10

V New Metrics 11

12 Multi-Scale Metrics M28–M32 11

VI Configuration 11

13 Scale Configuration 11

VII Rust API 12

14 Core Types 12

15 Universe 13

16 Bridges 13

17 Ladders 14

18 Clustering 14

19 Multi-Scale Engine Hook 15

VIII Acceptance Tests 16

20 Test Catalogue 16

IX Dependency Graph and Build Order 17

21 Updated Dependency Graph 17

22 Build Order 17

23 Test Count 18

X Implementation Handoff 18

24 Verification Sequence 18

25 Completion Criteria 18

Part I
Foundations

1 Purpose

ISLS currently observes a flat graph of entities at a single scale. All entities are peers; there is no notion of containment, hierarchy, or scale separation. This limits the system in three ways:

1. Patterns that manifest only at aggregate scales (e.g., sector rotations, regime transitions) are invisible to micro-level constraint extraction.
2. The $O(n^2)$ edge complexity of S-Scale (2000 entities, 70s per tick) demonstrates that flat observation does not scale.
3. Cross-domain observation (multiple independent systems in one ISLS instance) has no structural separation.

C18 resolves all three by introducing a hierarchical scale architecture in which the persistent graph is partitioned into *Hypercube Universes* at three scales, connected by *Bridges* (intra-scale coupling) and *Ladders* (inter-scale aggregation/disaggregation).

2 Axioms

Axiom 2.1 (Scale Separation)**.** Observable structure exists at multiple scales simultaneously. No single scale is sufficient to capture all discoverable constraint programs.

Axiom 2.2 (Scale Invariance of Substrate)**.** The PSP substrate — state space $\Sigma_0 = \mathbb{R}^5$, canonicalization, Run Descriptor, Evidence Chain, Crystal format, Gate cascade — is identical at every scale. Only the *population* of the state space (which entities, which edges, which observations) differs.

Axiom 2.3 (Directed Scale Hierarchy)**.** The scale hierarchy is strictly ordered: micro $\prec$ meso $\prec$ macro. Information flows upward via aggregation (Lift) and downward via disaggregation (Projection). No scale may depend on a scale above it for its internal dynamics.

Axiom 2.4 (Bridge Locality)**.** Bridges couple Hypercubes at the same scale only. Cross-scale coupling is handled exclusively by Ladders.

Axiom 2.5 (Crystal Scale Provenance)**.** Every crystal carries provenance indicating the scale at which it was discovered: micro, meso, or macro. A meso-crystal **MAY** reference micro-crystals as sub-evidence. A macro-crystal **MAY** reference meso-crystals.

3 Relationship to Existing Crates

Depends on	Provides to C18	Mechanism
`isls-types`	$\mathbb{R}^5$ state, Hash256	Foundation types
`isls-persist`	PersistentGraph, edges	Micro-scale substrate
`isls-topology` (C16)	Spectral clusters, Kuramoto r_k	Meso-scale cluster identification
`isls-extract`	inverse_weave	Per-scale constraint extraction
`isls-consensus`	Gate cascade	Per-scale crystallization
`isls-scheduler` (C15)	Spiral scheduler	Scale-adaptive tick resolution

Part II
Scale Architecture

4 The Three Scales

Definition 4.1 (Micro Scale $\mathcal{S}_\mu$)**.** The micro scale is the existing ISLS persistent graph $G = (V, E)$ where V is the set of observed entities and E the set of co-occurrence / correlation edges. Each vertex $v \in V$ carries a $\mathbb{R}^5$ embedding. This is the raw observation layer.

Definition 4.2 (Meso Scale $\mathcal{S}_m$)**.** The meso scale is a graph $G_m = (V_m, E_m)$ where each vertex $c \in V_m$ is a *cluster* of micro-entities and each edge $e \in E_m$ is an inter-cluster coupling. Clusters are identified by spectral analysis (Fiedler vector bisection, C16) and/or Kuramoto synchronization (C16).

Definition 4.3 (Macro Scale $\mathcal{S}_M$)**.** The macro scale is a graph $G_M = (V_M, E_M)$ where each vertex $u \in V_M$ is a *domain* or *regime* and each edge $e \in E_M$ is an inter-domain coupling. Domains are identified by second-order clustering: clustering the meso-clusters.

Remark 4.1. The three scales are not fixed at compile time. They are emergent: the system discovers how many clusters exist at the meso level and how many domains at the macro level. If the data has no meaningful cluster structure, G_m collapses to a single node and the meso/macro layers are trivial.

5 Hypercube Universe

Definition 5.1 (Hypercube Universe)**.** A Hypercube Universe $\mathcal{U}$ is a tuple

$$\mathcal{U} = \langle \texttt{id}, \texttt{scale}, G_\mathcal{U}, \Pi_\mathcal{U}, T_\mathcal{U}, \mathcal{P}_\mathcal{U}, \mathcal{B}_\mathcal{U}, \Sigma_\mathcal{U} \rangle$$

where:

- `id`: content-addressed identifier,
- `scale` $\in \{\mu, m, M\}$: scale level,
- $G_\mathcal{U} = (V_\mathcal{U}, E_\mathcal{U})$: the state space graph (directed, weighted),
- $\Pi_\mathcal{U}$: the set of $\mathbb{R}^5$ embeddings for all vertices,
- $T_\mathcal{U}$: the set of admissible transitions (CubeTube `Transition` trait),
- $\mathcal{P}_\mathcal{U}$: the active policy (CubeTube `Policy` trait),
- $\mathcal{B}_\mathcal{U}$: the 5D bounding box (`HyperBounds`) of the universe,
- $\Sigma_\mathcal{U}$: aggregate state vector of the universe (a single $\mathbb{R}^5$ point summarizing the collective state).

Definition 5.2 (Aggregate State)**.** The aggregate state of a Hypercube Universe is computed by an aggregation operator Agg:

$$\Sigma_\mathcal{U} = \mathrm{Agg}(\Pi_\mathcal{U}) = (\bar{p},\ \bar{\rho},\ \bar{\omega},\ \bar{\chi},\ r_\mathcal{U})$$

where $\bar{p}, \bar{\rho}, \bar{\omega}, \bar{\chi}$ are the weighted means of the entity embeddings (weighted by edge degree) and $r_\mathcal{U}$ is the Kuramoto order parameter of the universe's entities.

Definition 5.3 (HyperBounds)**.** A HyperBounds object defines the axis-aligned bounding box in $\mathbb{R}^5$:

$$\mathcal{B} = [\min_p, \max_p] \times [\min_\rho, \max_\rho] \times [\min_\omega, \max_\omega] \times [\min_\chi, \max_\chi] \times [\min_\eta, \max_\eta]$$

with operations:

- `contains`(σ): test whether $\sigma \in \mathcal{B}$,
- `volume`(): 5D hypervolume $\prod_{d=1}^{5}(\max_d - \min_d)$,
- `split`(d): bisect along dimension d, producing two child HyperBounds,
- `split_all`(): produce $2^5 = 32$ children (full subdivision),
- `merge`($\mathcal{B}'$): smallest enclosing box.

6 Transitions and Dynamics

Definition 6.1 (Scale Transition)**.** A scale transition is a deterministic map $T : \Sigma_{\mathcal{U}} \to \Sigma_{\mathcal{U}}$ that evolves the aggregate state of a universe. The canonical transition vocabulary is:

(i) `Identity`: $\Sigma \mapsto \Sigma$,
(ii) `Translate`(δ): $\Sigma \mapsto \Sigma + \delta$ for offset $\delta \in \mathbb{R}^5$,
(iii) `Scale`(α): $\Sigma \mapsto \alpha \cdot \Sigma$,
(iv) `PhaseAdvance`($\dot{\omega}$): increment the ω coordinate by $\dot{\omega} \cdot \Delta t$,
(v) `LerpToward`(Σ^*, γ): $\Sigma \mapsto \Sigma + \gamma(\Sigma^* - \Sigma)$,
(vi) `Decay`(τ): $\Sigma \mapsto \Sigma \cdot e^{-\Delta t/\tau}$.

All transitions **MUST** be deterministic under fixed (Σ, RD).

Definition 6.2 (Scale Dynamics)**.** Scale dynamics extend transitions with time and external input:

$$\Sigma_{k+1} = f(\Sigma_k, I_k, \Delta t, \mathrm{RD})$$

where I_k is an external input signal (e.g., observations flowing into the universe from a Ladder below). The dynamics function f **MUST** be a registered operator in the registry (C12).

Definition 6.3 (Scale Policy)**.** A scale policy $\mathcal{P}$ governs which transition is applied at each step within a universe:

(i) `Cyclic`: round-robin through a fixed transition sequence,
(ii) `Greedy`(Σ^*): select the transition that minimizes $\|\Sigma_{k+1} - \Sigma^*\|$,
(iii) `Balanced`: prefer less-visited regions of the state space (exploration bias),
(iv) `Reactive`: select transition based on current metric thresholds (e.g., switch to Decay when $r_{\mathcal{U}} < \theta_r$),
(v) `Custom`(f): user-defined deterministic function.

Policies **MUST** be deterministic. `Random` from CubeTube is excluded because ISLS forbids `rand`.

Part III
Bridges and Ladders

7 Dimensional Bridges

Definition 7.1 (Bridge)**.** A bridge $\mathcal{B}_{ij}$ is a directed coupling between two Hypercube Universes $\mathcal{U}_i$ and $\mathcal{U}_j$ at the same scale:

$$\mathcal{B}_{ij} = \langle \mathcal{U}_i, \mathcal{U}_j, w_{ij}, \tau_{ij}, \varphi_{ij}, \Pi_{ij} \rangle$$

where:

- $w_{ij} \in [0, 1]$: coupling strength (normalized),
- $\tau_{ij} \in \mathbb{N}_{\geq 0}$: propagation delay in ticks,
- $\varphi_{ij} \in [0, 2\pi)$: phase offset,

- Π_{ij}: the bridge policy governing when the coupling is active.

Definition 7.2 (Bridge Activation)**.** A bridge is active at tick k if and only if

$$\Pi_{ij}(\Sigma_{\mathcal{U}_i,k}, \Sigma_{\mathcal{U}_j,k}, \theta_{\text{bridge}}) = 1.$$

The canonical activation predicate is:

$$\Pi_{ij} = \mathbf{1}\left[\left\|\Sigma_{\mathcal{U}_i} - \Sigma_{\mathcal{U}_j}\right\| \leq \theta_{\text{prox}} \ \wedge \ r_{\mathcal{U}_i} \geq \theta_{r,\min} \ \wedge \ r_{\mathcal{U}_j} \geq \theta_{r,\min}\right]$$

i.e., the bridge activates when both universes are sufficiently coherent and their aggregate states are sufficiently proximate.

Definition 7.3 (Bridge Effect)**.** When bridge $\mathcal{B}_{ij}$ is active, the coupling modifies the target universe's dynamics:

$$\Sigma_{\mathcal{U}_j,k+1} = f_j(\Sigma_{\mathcal{U}_j,k}, I_k, \Delta t) + w_{ij} \cdot g(\Sigma_{\mathcal{U}_i,k-\tau_{ij}}, \varphi_{ij})$$

where g is the bridge transfer function:

$$g(\Sigma, \varphi) = \Sigma \cdot \cos(\omega(\Sigma) + \varphi)$$

extracting the phase-aligned component of the source universe's state.

Invariant 7.1 (Bridge Symmetry Breaking)**.** A bridge is directed: $\mathcal{B}_{ij}$ does not imply $\mathcal{B}_{ji}$. Bidirectional coupling requires two explicit bridges. This preserves causal directionality.

8 Scale Ladders

Definition 8.1 (Ladder)**.** A scale ladder $\mathcal{L}_{s\to s'}$ is a pair of operators connecting adjacent scales:

$$\mathcal{L}_{s\to s'} = (\text{Lift}_{s\to s'}, \text{Proj}_{s'\to s})$$

where $s \prec s'$ (e.g., $\mu \prec m$ or $m \prec M$).

8.1 Micro → Meso Ladder

Definition 8.2 (Micro-to-Meso Lift)**.** Given a spectral cluster assignment $c : V \to V_m$ (from C16's Fiedler bisection or Kuramoto synchronization), the lift operator constructs the meso graph:

$$\text{Lift}_{\mu\to m}(G, c) = G_m = (V_m, E_m, \Pi_m)$$

where:

- $V_m = \{c_1, \ldots, c_K\}$ is the set of clusters,
- $\Pi_m(c_i) = \text{Agg}(\{\Pi(v) : c(v) = c_i\})$ is the aggregate embedding of cluster c_i,
- $E_m(c_i, c_j) = \sum_{v\in c_i, w\in c_j} W_{vw}$ is the inter-cluster edge weight (sum of micro-edge weights between cluster members).

Definition 8.3 (Meso-to-Micro Projection)**.** The projection operator disaggregates a meso-level signal δ_m back to the micro level:

$$\text{Proj}_{m\to\mu}(\delta_m, c) = \left\{ \frac{\delta_{c(v)}}{|c(v)|} : v \in V \right\}$$

distributing the meso-level perturbation uniformly to all members of each cluster.

8.2 Meso → Macro Ladder

Definition 8.4 (Meso-to-Macro Lift)**.** A second-order clustering $d : V_m \to V_M$ groups meso-clusters into domains. The lift is structurally identical to $\text{Lift}_{\mu\to m}$:

$$\text{Lift}_{m\to M}(G_m, d) = G_M = (V_M, E_M, \Pi_M)$$

with domain-aggregate embeddings and inter-domain edge weights.

Definition 8.5 (Macro-to-Meso Projection)**.** Identical structure to $\text{Proj}_{m\to\mu}$, distributing macro-level signals to meso-clusters.

8.3 Ladder Consistency

Theorem 8.1 (Lift-Project Quasi-Inverse)**.** $\text{Proj}_{s'\to s} \circ \text{Lift}_{s\to s'}$ *is not the identity (it loses intra-cluster variation) but satisfies:*

$$\text{Agg}(\text{Proj}_{m\to\mu}(\text{Lift}_{\mu\to m}(G, c))) = \text{Agg}(G)$$

i.e., the global aggregate state is preserved under the lift-then-project round-trip.

Proof. Lift computes per-cluster weighted means. Proj distributes these means uniformly. The global weighted mean of the projected values equals the original global weighted mean because:

$$\frac{1}{|V|}\sum_{v\in V}\frac{\text{Agg}(c(v))}{|c(v)|}\cdot|c(v)| = \frac{1}{|V|}\sum_i \text{Agg}(c_i)\cdot|c_i| = \text{Agg}(G). \qquad \square$$

Invariant 8.1 (Ladder Direction)**.** Information flows upward (Lift) passively and continuously. Information flows downward (Projection) only when a crystal or alert at a higher scale triggers a disaggregation signal. This prevents downward information leakage from polluting micro-level observations.

Part IV
Multi-Scale Engine Integration

9 Per-Scale Macro-Step

Definition 9.1 (Scale-Partitioned Macro-Step)**.** At each engine tick k, the multi-scale macro-step executes:

(MS1) **Micro tick**: Execute the existing L0→L4 pipeline on G, producing micro-embeddings, micro-constraints, micro-crystals.

(MS2) **Lift** $\mu \to m$: Construct G_m from spectral clusters (C16) and micro-embeddings.

(MS3) **Bridge propagation (meso)**: For each active bridge $\mathcal{B}_{ij}$ at meso scale, apply coupling effect.

(MS4) **Meso tick**: Run constraint extraction and gate cascade on G_m, producing meso-constraints, meso-crystals.

(MS5) **Lift** $m \to M$: Construct G_M from meso-clusters.

(MS6) **Bridge propagation (macro)**: For each active bridge at macro scale, apply coupling effect.

(MS7) **Macro tick**: Run constraint extraction and gate cascade on G_M, producing macro-constraints, macro-crystals.

(MS8) **Downward projection (optional)**: If a macro-crystal triggers, project a disaggregation signal through $\mathrm{Proj}_{M\to m} \circ \mathrm{Proj}_{m\to\mu}$ to refine micro-level observation.

Requirement 9.1 (Scale Budget)**.** Each scale has an independent compute budget. If the meso or macro scale exceeds its budget, it is skipped for that tick. The micro scale is never skipped.

Requirement 9.2 (Scale-Aware Scheduling)**.** The spiral scheduler (C15) **MAY** assign different sub-step counts to different scales:

- Micro: $n_k^{(\mu)}$ sub-steps (existing behavior),
- Meso: $n_k^{(m)} = \max(1, n_k^{(\mu)}/\alpha_m)$ where $\alpha_m \geq 1$ is the meso decimation factor,
- Macro: $n_k^{(M)} = \max(1, n_k^{(\mu)}/\alpha_M)$ where $\alpha_M \geq \alpha_m$.

This ensures higher scales tick less frequently than lower scales.

10 Cluster Identification

Definition 10.1 (Spectral Bisection Clustering)**.** Using the Fiedler vector u_1 from C16's spectral decomposition:

1. Sort vertices by their Fiedler vector component $u_1(v)$.
2. Bisect at the median: $c_0 = \{v : u_1(v) \leq \tilde{u}_1\}$, $c_1 = \{v : u_1(v) > \tilde{u}_1\}$.
3. Recursively bisect each half until the spectral gap of a sub-cluster falls below $\theta_{\Delta,\min}$ or $|c_i| < n_{\min}$.

Definition 10.2 (Kuramoto Synchronization Clustering)**.** As a complementary method (or replacement when spectral decomposition is budget-constrained):

1. Run Kuramoto integration for N_{sync} steps (C16).
2. Group vertices whose phase difference $|\phi_i - \phi_j| < \theta_\phi$ into the same cluster.
3. Merge clusters smaller than $n_{\min}$ into their nearest neighbor cluster.

Requirement 10.1 (Deterministic Clustering)**.** Both clustering methods **MUST** produce identical results under identical (Σ, RD). Tie-breaking **MUST** use the canonical vertex ID ordering.

11 Multi-Scale Crystallization

Definition 11.1 (Scale-Tagged Crystal)**.** A crystal produced by the multi-scale engine carries an extended provenance:

$$C = \langle \ldots, \texttt{scale}, \texttt{universe_id}, \texttt{sub_crystal_ids}, \texttt{parent_crystal_ids}, \tau_{\text{multi}} \rangle$$

where:

- `scale` $\in \{\mu, m, M\}$: the scale at which this crystal was discovered,
- `universe_id`: the Hypercube Universe that produced it,
- `sub_crystal_ids`: list of crystal IDs at the scale below that constitute evidence for this crystal,
- `parent_crystal_ids`: list of crystal IDs at the scale above that reference this crystal,
- τ_{multi}: the multi-scale topological signature.

Definition 11.2 (Multi-Scale Topological Signature)**.** The multi-scale topological signature extends the hardened signature from C16:

$$\tau_{\text{multi}}(C) = \langle \tau_\mu, \tau_m, \tau_M, n_{\text{clusters}}, n_{\text{domains}}, n_{\text{bridges}}, n_{\text{active_bridges}}, \text{scale_coherence} \rangle$$

where:

- τ_μ, τ_m, τ_M: per-scale topological signatures (from C16),
- n_{clusters}: number of meso-clusters,
- n_{domains}: number of macro-domains,
- n_{bridges}: total bridges,
- $n_{\text{active_bridges}}$: bridges currently active,
- scale_coherence $= \frac{r_M + r_m + r_\mu}{3}$: average Kuramoto coherence across scales.

Theorem 11.1 (Cross-Scale Crystal Hierarchy)**.** *If a macro-crystal C_M references meso-crystals $\{C_{m,1}, \ldots, C_{m,k}\}$ and each $C_{m,i}$ references micro-crystals $\{C_{\mu,i,1}, \ldots\}$, then the entire hierarchy is independently verifiable: each crystal at each scale passes the 8-check formal validation independently, and the sub-crystal references are content-addressed.*

Proof. By construction: each crystal is produced by the same staged closure and gate cascade, regardless of scale. Content-addressing ensures reference integrity. The 8-check validation (content_address, dual_consensus, evidence_chain, free_energy, gate_kairos, immutability, operator_versions, por_trace) operates on the crystal's own fields, not on its scale. Therefore each level validates independently. □

Part V
New Metrics

12 Multi-Scale Metrics M28–M32

Definition 12.1 (M28 Cluster Count)**.** $M_{28} = |V_m|$: number of meso-clusters. Stability alert if M_{28} changes by more than 50% between consecutive ticks (cluster structure unstable).

Definition 12.2 (M29 Bridge Activity)**.** $M_{29} = n_{\text{active_bridges}}/n_{\text{bridges}}$: fraction of active bridges. Low values indicate decoupled universes; high values indicate strong inter-cluster coupling.

Definition 12.3 (M30 Scale Coherence)**.** $M_{30} = (r_M + r_m + r_\mu)/3$: average Kuramoto coherence across all three scales. Values near 1 indicate that structure is coherent at all levels; values near 0 indicate scale-dependent incoherence.

Definition 12.4 (M31 Lift Compression Ratio)**.** $M_{31} = |V_m|/|V|$: the ratio of meso-vertices to micro-vertices. This measures how much the hierarchy compresses. Typical healthy range: $0.01 \leq M_{31} \leq 0.2$.

Definition 12.5 (M32 Cross-Scale Crystal Rate)**.** M_{32}: number of crystals with `sub_crystal_ids` $\neq \emptyset$ per unit time. This measures how often the system discovers patterns that span multiple scales.

Part VI
Configuration

13 Scale Configuration

```
scale:
  enabled: true
  micro:
```

```
    # Micro scale uses existing engine config
  meso:
    enabled: true
    clustering_method: "spectral"   # or "kuramoto" or "hybrid"
    min_cluster_size: 5
    max_clusters: 50
    spectral_gap_threshold: 0.01
    phase_threshold: 0.3            # for Kuramoto clustering
    decimation_factor: 5            # alpha_m
    budget_ms: 50
  macro:
    enabled: true
    min_domain_size: 2
    max_domains: 10
    decimation_factor: 25           # alpha_M
    budget_ms: 20
  bridges:
    proximity_threshold: 0.5        # theta_prox
    coherence_threshold: 0.3        # theta_r_min
    max_delay_ticks: 10
  ladders:
    lift_method: "weighted_mean"    # or "median" or "kuramoto_order"
    projection_method: "uniform"    # or "weighted"
    downward_on_crystal: true       # project on macro crystal
```

Part VII
Rust API

14 Core Types

```
// isls-scale/src/lib.rs

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize)]
pub enum Scale { Micro, Meso, Macro }

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct HyperBounds {
    pub min: FiveDState,
    pub max: FiveDState,
}

impl HyperBounds {
    pub fn contains(&self, state: &FiveDState) -> bool;
    pub fn volume(&self) -> f64;
    pub fn split(&self, dimension: usize) -> (HyperBounds, HyperBounds);
    pub fn split_all(&self) -> [HyperBounds; 32];
    pub fn merge(&self, other: &HyperBounds) -> HyperBounds;
    pub fn from_points(points: &[FiveDState]) -> Option<HyperBounds>;
```

```
    pub fn center(&self) -> FiveDState;
}
```

15 Universe

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct HypercubeUniverse {
    pub id: Hash256,
    pub scale: Scale,
    pub vertex_ids: Vec<VertexId>,
    pub aggregate_state: FiveDState,
    pub bounds: HyperBounds,
    pub policy: ScalePolicy,
    pub kuramoto_r: f64,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub enum ScalePolicy {
    Cyclic { sequence: Vec<String> },
    Greedy { target: FiveDState },
    Balanced,
    Reactive { thresholds: BTreeMap<String, f64> },
    Custom { name: String },
}

pub fn build_universe(
    vertices: &[VertexId],
    embeddings: &BTreeMap<VertexId, FiveDState>,
    scale: Scale,
    policy: ScalePolicy,
) -> HypercubeUniverse;

pub fn compute_aggregate(
    embeddings: &BTreeMap<VertexId, FiveDState>,
    degrees: &BTreeMap<VertexId, f64>,
    phases: &[f64],
) -> FiveDState;
```

16 Bridges

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Bridge {
    pub source_id: Hash256,       // source universe
    pub target_id: Hash256,       // target universe
    pub weight: f64,              // coupling strength [0,1]
    pub delay_ticks: u64,         // propagation delay
    pub phase_offset: f64,        // phase alignment
    pub active: bool,             // current activation state
}
```

```
pub fn evaluate_bridges(
    universes: &BTreeMap<Hash256, HypercubeUniverse>,
    bridges: &mut [Bridge],
    config: &ScaleConfig,
);

pub fn apply_bridge_coupling(
    target: &mut HypercubeUniverse,
    source_history: &[FiveDState],    // ring buffer of past states
    bridge: &Bridge,
    tick: u64,
);
```

17 Ladders

```
pub fn lift_micro_to_meso(
    graph: &PersistentGraph,
    clusters: &BTreeMap<VertexId, usize>,    // vertex -> cluster_id
    embeddings: &BTreeMap<VertexId, FiveDState>,
) -> (Vec<HypercubeUniverse>, Vec<Bridge>);

pub fn lift_meso_to_macro(
    meso_universes: &[HypercubeUniverse],
    domains: &BTreeMap<usize, usize>,        // cluster_id -> domain_id
) -> (Vec<HypercubeUniverse>, Vec<Bridge>);

pub fn project_macro_to_meso(
    signal: &FiveDState,
    domain: &HypercubeUniverse,
    meso_universes: &[HypercubeUniverse],
    domain_assignment: &BTreeMap<usize, usize>,
) -> BTreeMap<Hash256, FiveDState>;

pub fn project_meso_to_micro(
    signals: &BTreeMap<Hash256, FiveDState>,
    clusters: &BTreeMap<VertexId, usize>,
    meso_universes: &[HypercubeUniverse],
) -> BTreeMap<VertexId, FiveDState>;
```

18 Clustering

```
pub fn spectral_bisection_cluster(
    fiedler_vector: &[f64],
    vertex_ids: &[VertexId],
    spectral_gap_threshold: f64,
    min_cluster_size: usize,
    max_clusters: usize,
) -> BTreeMap<VertexId, usize>;
```

```
pub fn kuramoto_phase_cluster(
    phases: &[f64],
    vertex_ids: &[VertexId],
    phase_threshold: f64,
    min_cluster_size: usize,
) -> BTreeMap<VertexId, usize>;

pub fn hybrid_cluster(
    fiedler: &[f64],
    phases: &[f64],
    vertex_ids: &[VertexId],
    config: &ScaleConfig,
) -> BTreeMap<VertexId, usize>;
```

19 Multi-Scale Engine Hook

```
pub struct MultiScaleState {
    pub meso_universes: Vec<HypercubeUniverse>,
    pub macro_universes: Vec<HypercubeUniverse>,
    pub meso_bridges: Vec<Bridge>,
    pub macro_bridges: Vec<Bridge>,
    pub cluster_assignment: BTreeMap<VertexId, usize>,
    pub domain_assignment: BTreeMap<usize, usize>,
    pub meso_crystals: Vec<SemanticCrystal>,
    pub macro_crystals: Vec<SemanticCrystal>,
}

pub fn multi_scale_tick(
    micro_state: &GlobalState,
    scale_state: &mut MultiScaleState,
    spectral: &SpectralDecomposition,
    kuramoto: &KuramotoState,
    config: &ScaleConfig,
) -> MultiScaleTickResult;

pub struct MultiScaleTickResult {
    pub meso_crystals: Vec<SemanticCrystal>,
    pub macro_crystals: Vec<SemanticCrystal>,
    pub metrics: MultiScaleMetrics,
    pub events: Vec<ScaleEvent>,
}

pub struct MultiScaleMetrics {
    pub m28_cluster_count: u64,
    pub m29_bridge_activity: f64,
    pub m30_scale_coherence: f64,
    pub m31_lift_compression: f64,
    pub m32_cross_scale_crystal_rate: f64,
```

```
}

pub enum ScaleEvent {
    ClusterSplit { cluster_id: usize, into: Vec<usize> },
    ClusterMerge { clusters: Vec<usize>, into: usize },
    BridgeActivated { bridge_idx: usize },
    BridgeDeactivated { bridge_idx: usize },
    DownwardProjection { from_scale: Scale, crystal_id: Hash256 },
    FixpointReached { scale: Scale },
}
```

Part VIII
Acceptance Tests

20 Test Catalogue

ID	Name	Procedure
AT-SC1	HyperBounds containment	Create bounds; test points inside and outside; verify correctness.
AT-SC2	HyperBounds subdivision	Split_all(); verify 32 children; verify no overlap; verify union = parent.
AT-SC3	Aggregate state conservation	Compute Agg; lift; project; compute Agg again; verify equal.
AT-SC4	Spectral clustering determinism	Cluster twice with same input; verify identical assignments.
AT-SC5	Kuramoto clustering	Run Kuramoto to convergence; cluster by phase; verify synchronized entities are co-clustered.
AT-SC6	Bridge activation	Set two universes with proximate states and high coherence; verify bridge activates. Set distant states; verify bridge inactive.
AT-SC7	Bridge coupling effect	Activate bridge; verify target universe state is modified by source.
AT-SC8	Ladder lift-project round-trip	Lift micro→meso; project meso→micro; verify global aggregate preserved.
AT-SC9	Multi-scale crystal provenance	Run S-Basic with scale enabled; verify meso-crystals contain sub_crystal_ids referencing micro-crystals.
AT-SC10	Scale-disabled passthrough	Set scale.enabled=false; verify behavior identical to pre-extension.
AT-SC11	Budget fallback	Set meso budget to 1ms on large graph; verify meso tick is skipped and logged.
AT-SC12	Cross-scale crystal validation	Validate a macro-crystal and all referenced sub-crystals; verify all pass 8-check.

AT-SC13	Multi-scale metrics	Run 100 ticks with scale; verify M28–M32 are populated and non-zero.
AT-SC14	Downward projection	Trigger a macro-crystal; verify downward projection event and micro-level signals.
AT-SC15	Policy determinism	Run Greedy and Reactive policies; verify deterministic under same RD.

Part IX
Dependency Graph and Build Order

21 Updated Dependency Graph

```
isls-types        <-- (all crates)
    ^
isls-registry     <-- (all operator crates)
    ^
isls-observe  <-- isls-persist <-- isls-topology (C16)
                        ^                ^
                        |          isls-extract
                        |                ^
                        |          isls-scale (C18: uses topology + persist)
                        |                ^
                isls-consensus <-- isls-carrier
                        ^
                isls-archive <-- isls-morph
                        ^
                isls-manifest
                        ^
                isls-capsule
                        ^
                isls-scheduler (uses scale events for adaptive ticking)
                        ^
                isls-engine (+ multi_scale_tick integration)
                        ^
                isls-store (C17)
                        ^
                isls-harness (+ scale tests)
                        ^
                isls-cli (+ scale metrics in reports)
```

22 Build Order

1. `isls-scale` (C18): depends on `isls-types`, `isls-persist`, `isls-topology` (C16). No new external dependencies.
2. Engine integration: wire `multi_scale_tick` into the engine loop after the micro tick.
3. Scheduler integration: feed scale events into scheduling function.

4. Crystal format extension: add scale provenance fields.
5. CLI/report integration: add M28–M32 to metric dashboard and HTML report.
6. Harness: add AT-SC1 through AT-SC15.

23 Test Count

Phase	New Tests	Cumulative
Core (C1–C9)	130	130
Phase 1 (C10–C15)	31	161
Phase 2 (C16–C17)	20	181
Phase 3 (C18)	15	$\geq$ 196

Part X
Implementation Handoff

24 Verification Sequence

```
# Build
cargo build --workspace --release 2>&1 | grep -c warning  -> 0

# Test
cargo test --workspace -- --test-threads=1  -> >= 196 passed

# S-Basic with multi-scale
rm -rf ~/.isls
cargo run --bin isls --release -- ingest --adapter synthetic --scenario S-Basic
cargo run --bin isls --release -- run --mode shadow --ticks 100

# Verify multi-scale output
# - Meso universes created (cluster count > 1)
# - At least 1 meso-crystal with sub_crystal_ids
# - M28-M32 populated in metrics
# - Bridges evaluated (some active, some inactive)

cargo run --bin isls --release -- validate formal
# All crystals pass (micro + meso + macro)
```

25 Completion Criteria

Phase 3 is complete when:

1. `isls-scale` compiles with zero warnings.
2. All AT-SC tests pass.
3. S-Basic produces crystals at micro and meso scales.
4. M28–M32 metrics appear in reports.
5. Scale disabled produces identical results to pre-extension.
6. Multi-scale crystals reference sub-crystals and pass validation.
7. Total tests $\geq$ 196.

Generated for ISLS Extension Phase 3 v1.0.0.
Hypercube Universes. Dimensional Bridges. Scale Ladders.
Deterministic, append-only, replay-verified.